

MTH6161 Neural Networks and Deep Learning

MNIST Autoencoder Mini-Project

Redwan Munshi
Queen Mary University of London

May 2026

Task and Aim

The aim of this mini-project is to build an autoencoder, train it on the MNIST dataset, and analyse the features extracted by the bottleneck layer. An autoencoder is suitable for this task because it learns a compressed representation of an input image without using class labels during training.

This is different from standard supervised classification. The network is not trained to predict the digit label. Instead, it is trained to reproduce the input image as closely as possible. After training, the latent vectors are treated as extracted features and are analysed using PCA, reconstruction errors for each digit class, and interpolation in the latent space.

Dataset and Preprocessing

The dataset used is MNIST, which contains greyscale images of handwritten digits from 0 to 9. Each image has size 28×28 , so each image contains 784 pixel values. The notebook uses the standard MNIST split, with 60,000 training images and 10,000 test images.

The images are converted to tensors and scaled so that the pixel values lie between 0 and 1. During training, the digit labels are not used, because the autoencoder is trained to reconstruct its input rather than predict a class label. The labels are only used after training to interpret the latent space and compare reconstruction errors across digit classes.

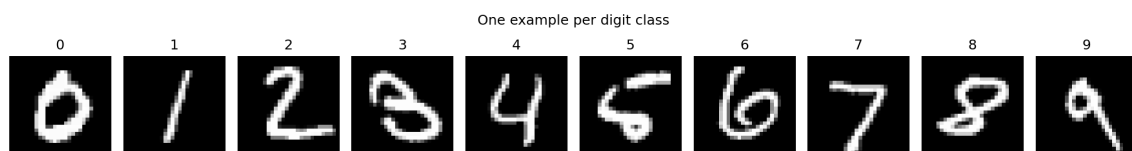


Figure 1: One example from each digit class in MNIST.

Architecture

The model used is a fully connected autoencoder. Each 28×28 image is flattened into a vector of length 784 before it is passed into the network. The encoder reduces the input size from 784 to 256, then to 64, and finally to a latent vector of size 16. ReLU activation functions are used between the linear layers. The decoder then maps the 16-dimensional latent vector back to 64,

then 256, and finally back to 784 output values. A sigmoid activation function is used at the end of the decoder so that the reconstructed pixel values are also between 0 and 1.

The bottleneck layer is the main part of the project, because it forces the network to keep only the most useful information needed to reconstruct the image. In this project, the 16 numbers in the bottleneck layer are used as the extracted features.

Training Strategy

The model is trained using mean squared error loss. For one image, the loss compares the original image x and the reconstructed image $\hat{x}$:

$$\text{MSE}(x, \hat{x}) = \frac{1}{784} \sum_{i=1}^{784} (x_i - \hat{x}_i)^2.$$

The Adam optimiser is used with learning rate 0.001. The batch size is 128 and the model is trained for 25 epochs. In each training step, the model first produces a reconstruction of the input image. The MSE loss is then calculated, backpropagation is applied, and the optimiser updates the model parameters.

The training loss decreased over the epochs, which shows that the autoencoder learned to reconstruct the MNIST images. The final training MSE was 0.017365. The average test MSE was 0.017603. The test loss is important because it checks whether the model can reconstruct images that were not used for training.

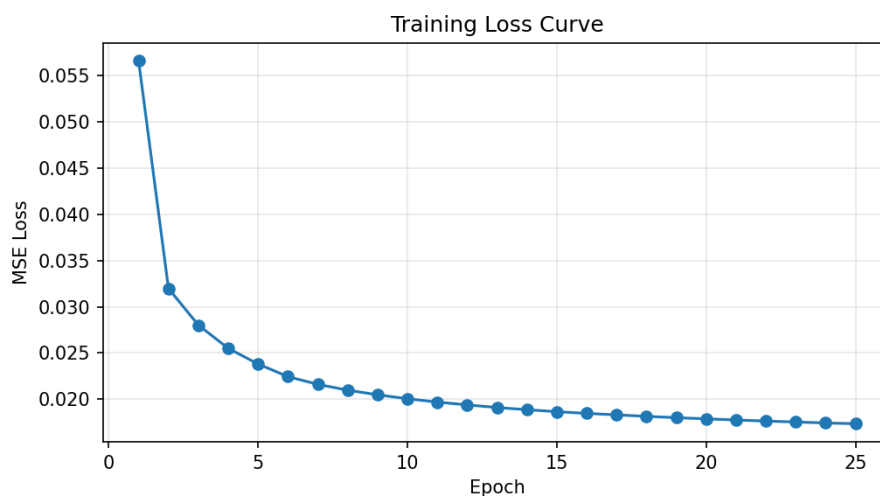


Figure 2: Training loss over 25 epochs.

Results and Analysis

Reconstructed Images

The first result is the visual comparison between original test images and reconstructed images. The reconstructions keep the main shape of the digits, which shows that the latent layer is storing useful information about the input image.

The reconstructed images are not perfect. Some edges are smoother and some details are lost. This is expected because the image is compressed from 784 pixel values to only 16 latent

values. The model has to remove some detail and keep the features that are most useful for reconstruction.

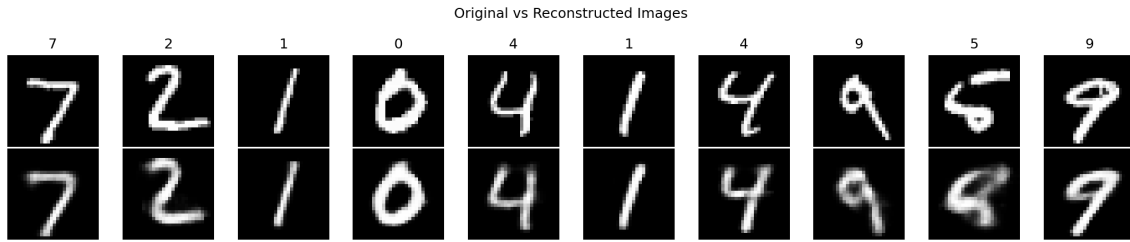


Figure 3: Original test images compared with reconstructed images.

PCA of the Latent Features

After training, the encoder was used to convert the test images into 16-dimensional latent vectors. PCA was then applied to reduce these latent vectors to two dimensions for plotting. The first principal component explained 28.64% of the variance, and the second principal component explained 23.82% of the variance.

The PCA plot shows that the latent space contains useful structure. Points from the same digit class often appear close together, even though the model was not trained with labels. This suggests that the autoencoder has learned features related to digit shape. However, the classes are not completely separated. This is also expected, because the training objective is reconstruction rather than classification. Digits with similar shapes, such as 4 and 9 or 3 and 8, can overlap in the latent space.

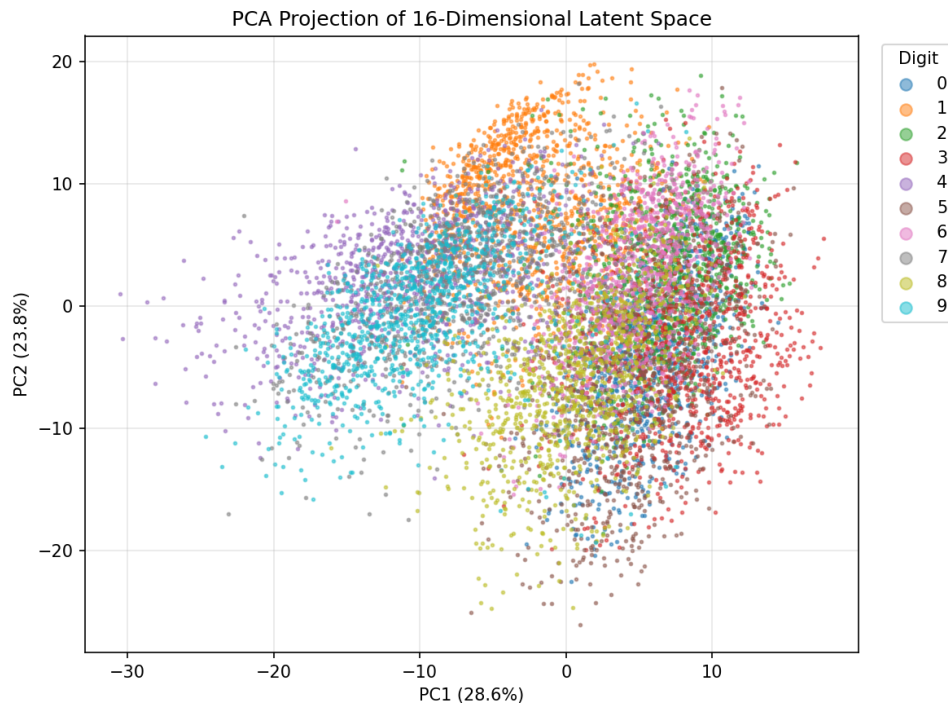


Figure 4: PCA projection of the 16-dimensional latent vectors.

Reconstruction Error by Digit

The reconstruction error was also calculated separately for each digit class. This gives a simple way to check whether some digits are harder for the autoencoder to reconstruct than others.

The easiest digit to reconstruct was 1, while the hardest digit was 8. Differences between the digits can happen because some digits have simpler shapes and less variation in handwriting. Digits with more variation, or digits that are often written in different styles, can be harder to reconstruct accurately.

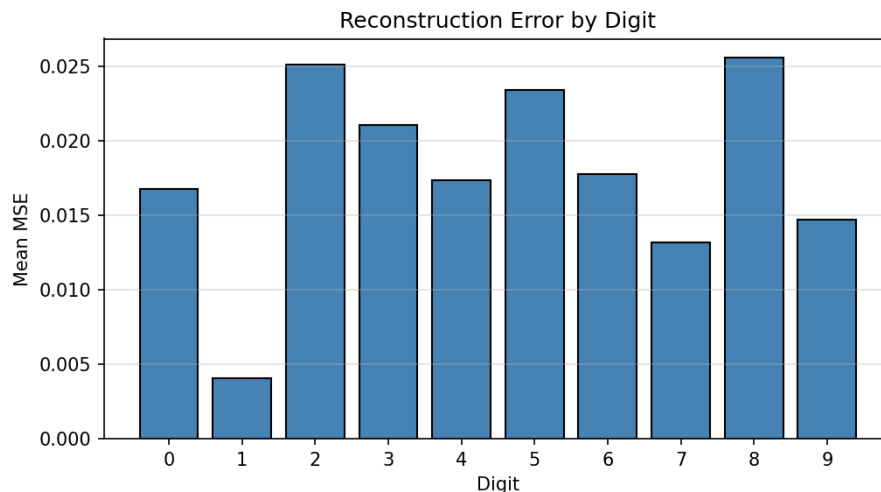


Figure 5: Mean reconstruction error for each digit class.

Latent Space Interpolation

Latent interpolation was used to test whether the latent space changes smoothly. Two test images were chosen, one from digit 4 and one from digit 9. Their latent vectors were mixed gradually, and the decoder was used to turn each mixed vector back into an image.

The interpolation shows that the output changes gradually from one digit shape to the other. This suggests that nearby points in the latent space usually correspond to similar reconstructed images. This is useful evidence that the bottleneck layer has learned a meaningful representation, rather than just memorising unrelated codes for each image.

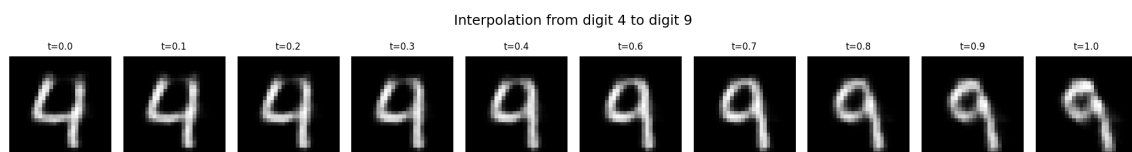


Figure 6: Interpolation in latent space between a digit 4 and a digit 9.

Discussion

Overall, the autoencoder works well for the MNIST reconstruction task. The low test MSE and the visual reconstructions show that the model can keep the main digit shape after compressing the image into a 16-dimensional latent vector. The PCA plot and interpolation also show that the latent vectors contain useful extracted features.

There are some limitations. The model is fully connected, so it does not directly use the spatial structure of images. A convolutional autoencoder could be better for image data because it can use local pixel patterns. Also, PCA only shows a two-dimensional view of a 16-dimensional space, so it cannot show all the information in the latent vectors. Finally, this is a standard autoencoder, not a variational autoencoder, so it should not be treated as a full generative model.

Conclusion

This project built and trained an autoencoder on MNIST. The model compressed each image from 784 pixel values to 16 latent features and reconstructed the image from those features. The results show that the network learned useful compressed representations of handwritten digits. The reconstructions, PCA plot, per-digit reconstruction errors, and interpolation all support the conclusion that the bottleneck layer extracts meaningful features from the input images.

References

- [1] Y. LeCun, C. Cortes, and C. J. C. Burges, *The MNIST Database of Handwritten Digits*. Available online.
- [2] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [3] M. Hanada, *QMUL MTH6161/MTH767P Lecture Notes*, 2025/2026.